A Practical Guide to Encoder decoder architecture

TOPIC -- ENCODER DECODER ARCHITECTURE

-- 01 --

Tokenization As the transformer architecture natively consists of operations over numbers (matrix multiplications, dot products, activation functions) rather than over text, there must first be a mapping from any input text to some numerical representation. This happens in three steps. First, the input text is treated by a preprocessor, which performs both textual transformations and splits the text into coarse-grained segments called pretokens. The latter is referred to as pretokenization. Second, each pretoken is segmented further into tokens by a tokenizer that expects to only see pretokens output by its preprocessor. Each token it produces is a string of one or more characters belonging to a finite set of strings called the vocabulary V {\displaystyle V} . Third, because the vocabulary is finite and known beforehand, each token can be assigned an integer identifier, and this mapping is applied to the sequence of tokens to represent any input text as a numerical sequence. Since this mapping is bijective, the output side can produce a sequence of integer identifiers which can then be turned back into tokens. After undoing some of the preprocessing, the result is again legible text. Training a tokenizer (sometimes referred to as vocabularization) means finding a suitable vocabulary V {\displaystyle V} , but also learning how to use it, since any given string s {\displaystyle s} of length | s | {\displaystyle |s|} has 2 | s | − 1 {\displaystyle 2^{|s|-1}} hypothetical segmentations, some of which containing segments that are not in the vocabulary. The most important hyperparameter during vocabularization is the vocabulary size | V | {\displaystyle |V|} : when it is small, the learned vocabulary generally consists of characters and smaller strings, and words will be segmented into many tokens. At larger sizes, it becomes affordable to dedicate tokens to full words, although depending on the preprocessor and tokenizer, it is not necessarily the case that large vocabularies will always use the largest token(s) available to segment a word. Because tokens are not always full words, they may also be referred to as subwords and tokenization algorithms may be referred to as subword tokenizers. This is also to differentiate these systems from traditional terminology used in older information retrieval and natural language processing systems, where "tokenization" was used to denote what is today called "pretokenization" (very crudely: splitting into words). In tokenizers that produce tokens that are not part of the vocabulary, a special token that does belong to the vocabulary is used as a generic stand-in, written as "[UNK]" for "unknown". In principle, any string could be hidden by such

an [UNK]. Indeed, in information retrieval, pretokenizers were themselves used as tokenizers (and also called "tokenizers") with a word-level vocabulary that contained an [UNK]. Commonly used subword tokenization algorithms are byte pair encoding (BPE) and the unigram language model (ULM), which each include a vocabularization algorithm and a dedicated segmentation algorithm. There also exist several segmentation algorithms that require no learning and can be applied given a vocabulary (produced by BPE or ULM, for example), like greedily recognising tokens in a pretoken by moving through it left-to-right. Well-known software implementations of subword tokenizers are Hugging Face's tokenizers Python package implemented in Rust, and the sentencepiece Python package implemented in C++. The latter package is named as such because one of its configuration options allows disabling the built-in pretokenizer, hence effectively making entire sentences a pretoken and thus having the tokenizer see entire sentences, rather than individual words.

-- 02 --

Like earlier seq2seq models, the original transformer model used an encoder–decoder architecture. The encoder consists of encoding layers that process all the input tokens together one layer after another, while the decoder consists of decoding layers that iteratively process the encoder's output and the decoder's output tokens so far. The purpose of each encoder layer is to create contextualized representations of the tokens, where each representation corresponds to a token that "mixes" information from other input tokens via self-attention mechanism. Each decoder layer contains two attention sublayers: (1) cross-attention for incorporating the output of encoder (contextualized input token representations), and (2) self-attention for "mixing" information among the input tokens to the decoder (i.e. the tokens generated so far during inference time). Both the encoder and decoder layers have a feed-forward neural network for additional processing of their outputs and contain residual connections and layer normalization steps. These feed-forward layers contain most of the parameters in a transformer model.

-- 03 --

Seq2seq models with attention (including self-attention) still suffered from the same issue with recurrent networks, which is that they are hard to parallelize, which prevented them from being accelerated on GPUs. In 2016, decomposable attention applied a self-attention mechanism to feedforward networks, which are easy to parallelize, and achieved SOTA result in textual entailment with an order of magnitude fewer parameters than LSTMs. One of its authors, Jakob Uszkoreit, suspected that attention without recurrence would be sufficient for

language translation, thus the title "attention is all you need". That hypothesis was against conventional wisdom at the time, and even his father Hans Uszkoreit, a well-known computational linguist, was skeptical. In the same year, self-attention (called intra-attention or intra-sentence attention) was proposed for LSTMs. On 2017-06-12, the original (100M-parameter) encoder–decoder transformer model was published in the "Attention is all you need" paper. At the time, the focus of the research was on improving seq2seq for machine translation, by removing its recurrence to process all tokens in parallel, but preserving its dot-product attention mechanism to keep its text processing performance. This led to the introduction of a multi-head attention model that was easier to parallelize due to the use of independent heads and the lack of recurrence. Its parallelizability was an important factor to its widespread use in large neural networks.

where the softmax is applied over each of the rows of the matrix. The number of dimensions in a query vector is query size d query {\displaystyle d_{\text{query}}} and similarly for the key size d key {\displaystyle d_{\text{key}}} and value size d value {\displaystyle d_{\text{value}}} . The output dimension of an attention head is its head dimension d head {\displaystyle d_{\text{head}}} . The attention mechanism requires the following three equalities to hold: ■ seq, key = ■ seq, value , d query = d key , d value = d head {\displaystyle \ell _{\text{seq, key}}=\ell _{\text{seq,
value}},\;d_{\text{query}}=d_{\text{key}},\;d_{\text{value}}=d_{\text{head}}} but is otherwise unconstrained. If the attention head is used in a self-attention fashion, then X query = X key = X value {\displaystyle X_{\text{query}}=X_{\text{key}}=X_{\text{value}}} . If the attention head is used in a cross-attention fashion, then usually X query ≠ X key = X value {\displaystyle X_{\text{query}}\neq X_{\text{key}}=X_{\text{value}}} . It is theoretically possible for all three to be different, but that is rarely the case in practice.